CERW (Criteria-Based Evaluation & Review Workspace) - Project Summary

This document provides a highly optimized, token-efficient technical overview of the CERW project. It covers the system architecture, database state schema, grading math, and core application workflows.

---

1. Project Overview & Architecture

CERW is a single-page, responsive web application designed for academic programs aligning with the Marco Nacional de Cualificaciones (MNC) of Colombia and SENA guidelines.

Technology Stack:

Frontend: *Single-page Vanilla HTML5, CSS3 (Modern dark-theme dashboard with Outfit & JetBrains Mono typography), and Vanilla ES6 JavaScript (app.js).*

Database/Persistence: Persistent, local browser state utilizing the localStorage API. No backend required.

AI Engine: *Client-side integration with the Google Gemini API (gemini-3.1-flash-lite) using direct HTTP requests.*

---

2. Core State Schema (localStorage)

The application persists all data under a single local storage key "cerw_state". The state structure is defined below:

```
{
  "theme": "dark" | "light",
  "activeTab": "dashboard" | "grading" | "mnc-suite" | "manager-suite",
  "selectedProgram": "Name of Active Program",
  "selectedSignature": "Name of Active Subject/Module",
  "selectedEvent": "Name of Active Evidence/Milestone",
  "geminiApiKey": "user-saved-gemini-api-key",
  "managerSubjects": [
    "Razonamiento cuantitativo",
    "Ingles",
    "Comunicacion oral y escrita",
    "Competencias digitales"
  ],
  "programs": {
    "Program Name": {
      "level": "Técnico Laboral (MNC)",
      "cuoc": "CUOC Occupational 4-digit code and title",
      "signatures": {
        "Signature/Subject Name": {
          "students": [
            { "id": "S1", "name": "Student Name", "tokens": 3 }
          ],
          "criteria": {
            "C1": {
              "name": "Competency Name: Description of performance",
              "type": "Core" | "Advanced" | "Transversal",
              "element": "Parent Performance Element"
            }
          },
          "events": ["Event 1 Name", "Event 2 Name"],
          "activeCriteriaByEvent": {
            "Event 1 Name": ["C1", "C2", "T1"]
          }
        }
      }
    }
  },
  "evaluations": {
    "StudentID_CriterionID_EventName": {
      "state": "Pending" | "Met" | "Not Met" | "Disputed",
      "defense": "Optional student appeal justification text",
```

```
      "timestamp": "ISO Date string"
    }
  }
}
```

---

3. Colombian Accordance Grading Rules (The Math)

Grading calculations strictly map binary competency evaluation to a standard Colombian academic decimal range of 1.0 to 5.0 (passing grade is 3.0).

Categories of Criteria:

1. ***Core (Obligatorias):*** *Indispensable competencies. If any Core criterion is failed, the grade is capped in the failing range (less than 3.0).*

2. ***Advanced (Especializadas):*** *Elevate grades to superior levels (3.1 to 5.0).*

3. ***Transversal (Transversales):*** *Generic work/soft skills.*

Math Logic:

For a given student under the active module, we check:

Core_Criteria_Total = Number of active Core criteria

Core_Criteria_Met = Number of met Core criteria

Advanced_Criteria_Total = Number of active Advanced criteria

Advanced_Criteria_Met = Number of met Advanced criteria

Case A: If Core_Criteria_Met is less than Core_Criteria_Total (Failing Grade)

The student has failed one or more Core criteria. The final grade is proportionally mapped between 1.0 and 2.9:

Formula: Grade = 1.0 + (1.9 *(Core_Criteria_Met / Core_Criteria_Total))*

Case B: If all Core criteria are met (Passing Grade)

If Advanced_Criteria_Total is 0:

The final grade is a perfect 5.0.

If Advanced_Criteria_Total is greater than 0:

The final grade is mapped between 3.0 and 5.0 based on the percentage of advanced criteria met:

Formula: Grade = 3.0 + (2.0 *(Advanced_Criteria_Met / Advanced_Criteria_Total))*

---

4. Key Workflows & Features

A. Two-Dimensional Grading Matrix

Displays students as rows and active evaluation criteria as columns.

Interactive Toggling: *Clicking a cell cycles through states: Pending (Yellow) -> Met (Green) -> Not Met (Red) -> Disputed (Purple).*

Highlighters: Hovering over cells highlights the corresponding student row and criterion column for quick scanning.

B. Student Appeals & Token System

Simulator View: *Provides a collapsible mobile dock representing the student portal view.*

Re-evaluation Tokens: Students start with 3 tokens. Selecting a "Not Met" cell allows them to spend 1 token to enter a written appeal defense, moving the status to Disputed.

Teacher Decision: *Teachers review disputes in the Priority Inbox. Approving changes the status to Met (token is refunded). Maintaining keeps it at Not Met (token is lost).*

C. MNC Suite (AI Extraction)

API Extraction: Users upload a Technical Sheet (.txt or .pdf). The Gemini API processes it, automatically downgrades high-level verbs to Technical standards (implement, test, execute), and returns a structured JSON payload.

SENA Grammatical Redaction Rule: *Criteria are strictly renamed using the schema:*

"[Competency]: [Verbo en Infinitivo] + [Objeto] + [Condición de Referencia/Calidad]"

Limit: Enforces 3 to 5 criteria per subject to prevent layout overflow.

D. Manager Suite (Harmonization)

Multi-Subject Alignment: *Managers enter a program name and select subjects (e.g., Quantitative Reasoning, English, Digital Skills).*

Unified Plan Creation: Gemini aligns the uploaded catalog and returns localized criteria for all selected subjects.

Database Seeding: *Saving generates the new program, initializes rosters (5 mock students), seeds default evaluation records, and routes to the Matrix.*

---

5. Main Script Reference (app.js)

calculateGradesAndMetrics(): Recomputes overall grades, class averages, active appeal lists, and risk metrics.

refreshUI(): Dynamically redraws the matrix table, metric cards, dashboard warnings, histograms, and student portals.

analyzeWithGeminiAPI(fileData, mimeType, filename, isText): Formulates prompts and communicates with the Google API key endpoints.

parseMNCDocument(text): Local offline fallback regex parser for text files.

harmonizeProgramWithGemini(): Curricular alignment processor for multi-subject configurations in the Manager view.

- executeLocalOfflineHarmonization(...): Offline fallback module for manager harmonization using pre-designed curricular templates.